



Computer Networks Last Night Notes

CSAI321 – All 6 Lectures Covered



Sab kuch yahan hai, ab PDF kholne ki zaroorat nahi 😎

By: Padhai Mode ON 🧠 / Date: KAL TEST HAI BHAI



L1: Internet



L2: Data Moving



L3: Layers



L4: Devices



L5: HTTP & P2P



L6: HTTPS & TLS



MCQ Practice

Lecture 1



How the Internet Works



Internet kya hai?

Internet ek **network of networks** hai – matlab hazaron alag-alag networks jo ek doosre se connected hain. Ye sirf "kisi aur ka computer" nahi – ye millions of physical servers hain jo **data centers** mein rakhe hain aur high-speed cables se jude hain.



Fun fact: 2025 mein 597+ submarine cable systems hain jo continents ko jodti hain. Inhe khatre? Fishing anchors aur sharks! 🐟



Internet ka Structure: Edge vs Core



Network Edge

Woh devices jo internet use karte hain – tumhara phone, laptop, server. End systems bhi kehte hain inhe.



Network Core

Routers ka jaal jo data forward karta hai. Directly use nahi hota – bas data ek jagah se doosri jagah paas karta hai.



Network Types (Access Networks)

Type	Example	Range
LAN – Local Area Network	Ghar/Office ka WiFi	Chhoti jagah
MAN – Metropolitan Area Network	City-wide network	Ek sheher
WAN – Wide Area Network	Internet itself	Poori duniya
PAN – Personal Area Network	Bluetooth headphones	1-2 meter



AWS Vocabulary — EXAM MEIN AATA HAI!



Region

Ek geographic area jisme multiple data centers hain. Eg: Mumbai Region. 30+ globally hain.



Availability Zone (AZ)

Region ke andar ek ya zyada data centers.
Isolated fault domain – ek fail ho toh doosra chal sake.



Edge Location

CDN endpoints jo user ke paas content cache karte hain. Latency kam karta hai! d/s mein d kam hoti hai.



VPC

Virtual Private Cloud – AWS ke andar tumhara apna private software-defined network. Tum IP ranges, subnets control karo.



Instagram tap karo toh kya hota hai?

Thumb tap → App opens → WiFi/4G → Router → ISP → Internet Core → AWS/Meta servers → Data wapas tumhare paas → Reel plays! 🎬

Ye poora process milliseconds mein hota hai. Yahi is course ka saara magic hai! 🪄



Latency Intuition

Latency = kitna time lagta hai ek request ke response mein aane mein. Zyada hops = zyada latency. Edge locations isi liye hain – data ko user ke paas lao taaki delay kam ho.



Yaad rakho: Internet ek single entity nahi hai – ye lakhs of networks ka federation hai. Koi ek organization control nahi karti. ISPs, companies, governments sab mile hue hain.

 *Internet = Network of Networks. Edge = end users/servers. Core = routers that forward packets.*

1

Lecture 2 Moving Data Through the Core

Circuit Switching vs Packet Switching — Dadi wali call story!

Circuit Switching (OLD)

Pehle poori path reserve hoti thi. Traditional telephone model.

Dadi 10 sec chup rahe → capacity

waste

hoti thi!

- ✓ *Guaranteed bandwidth — no surprises*
- ✓ *Predictable, stable*
- ✗ *Wasteful for bursty data*
- ✗ *Idle capacity = wasted capacity*

Packet Switching (INTERNET)

Data packets mein toot jaata hai. Har packet alag route le sakta hai.

Tum chup rahe → koi packet nahi → capacity
free!

- ✓ *Efficient*
- ✓ *Resilient (alternate routes)*
- ✗ *No built-in guarantees*
- ✗ *Packets delay ya drop ho sakte hain*

 *Packet anatomy: Header (source+destination address) + Payload (actual data).
Packets independent routes le sakte hain — same destination ke liye!*

Store-and-Forward Rule — SUPER IMPORTANT!

Router poora packet receive karta hai PEHLE → phir aage forward karta hai.

Rule: Ek bhi bit forward nahi hoti jab tak poora packet nahi aa jaata!

Fayda: Har router sirf next hop jaanta hai — poora path jaanna zaruri nahi.

Statistical Multiplexing

Koi reservation nahi. Link sirf tab use hoti hai jab packet actually jaana chahta ho. 2 superpowers milte hain:

 **Efficiency:** Traffic bursty hoti hai. Sab ek saath rarely burst karte hain. Ek link many users share kar sakte hain.

 **Resilience:** Path fail ho toh doosra le lo! Kuch "reserved" nahi tha toh kuch khoega nahi.

Delay ke 4 Types — Toll Booth Caravan Analogy!

Cars (packets) → Toll Booth (router/transmission) → Highway (propagation). Har packet inhe experience karta hai:

Delay	Kya hai?	Formula	Exam mein?
 Processing	Router header padhta hai, bit errors check karta hai	~microseconds	Ignore karo
 Queuing	Queue mein wait (behind other packets)	Variable	Calculate nahi ✗
 Transmission	Sare bits ko link pe push karna (Toll Booth time)	L/R	CALCULATE ✓
 Propagation	Bit ka physically travel karna (Highway time)	d/s	CALCULATE ✓

▴ Transmission Delay (Toll Booth = kitna time bits push karne mein laga)

$$\text{Delay}_{\text{trans}} = L / R$$

L = Packet size (bits) | R = Link rate / bandwidth (bits/sec)

▴ Propagation Delay (Highway = bit kitna door travel karta hai)

$$\text{Delay}_{\text{prop}} = d / s$$

d = Distance (meters) | s = Propagation speed ($\sim 3 \times 10^8$ m/s in fiber/air)



EXAM SCOPE NOTE: Sirf Transmission (L/R) aur Propagation (d/s) calculate karne hain. Queuing delay ka koi maths OUT OF SCOPE hai! Simple plug-in calculations only.



Quick Example

Q: 8000 bits ka packet, 1 Mbps link pe transmit karne mein kitna time?

Ans: $L/R = 8000 / 1,000,000 = 0.008s = 8ms$ ✓



Throughput & Bottleneck

End-to-end speed = **bottleneck link** ki speed — yaani poore path mein **sabse slow link**.



Chain ki mazbooti = sabse kamzor kadi

Throughput = $\min(R_1, R_2, R_3, \dots)$ — minimum link rate

ISP 100 Mbps de raha hai lekin 10 Mbps ki link beech mein hai → throughput = 10 Mbps!



Packet Loss

Jab router ke buffers bhar jaate hain (queue full) → naye packets **drop** ho jaate hain. TCP baad mein isse handle karta hai (Week 5-6 mein padhoge).

2

Lecture 3



Protocol Layers — OSI & TCP/IP



Is lecture ka ek main idea: Network itna complicated hai ki ek cheez ki tarah design/debug karna impossible hai. Layering ka trick hai — har layer ek kaam karti hai, upar-neeche wali se baat karti hai, baaki layers ki chinta nahi.



Layering ka idea — Letter Analogy

Tum ek letter likhte ho, envelope mein daalo, postal service ko dete ho. Kya postman ko letter ka content pata hai?

NAHI! Usse sirf address chahiye. Tumhe van/flight routes ki zaroorat nahi. Dono apna kaam karte hain bina doosre ka kaam samjhe — letter phir bhi pohonch jaata hai! 📧

Layering ke 4 fayde:

- ✅ Har layer ek problem solve karti hai — alag se samjha ja sakta hai
- ✅ Ek layer replace kar sako bina doosri ko touch kiye (WiFi → Ethernet, kuch change nahi)
- ✅ Alag vendors ki layers bhi saath kaam karti hain (interoperability)
- ✅ Bug aaye toh specific layer mein dhundo — poora system nahi

📦 Encapsulation — Nesting Dolls / Gift Wrapping

Sender side (down the stack): Har layer apna header add karti hai — upar wali layer ke data ke bahar wrap karti hai.

Receiver side (up the stack): Har layer apna header strip karti hai (decapsulation) aur upar bhejti hai.

📦 Gift wrapping: Paper laga → Box mein daala → Ribbon bandha. Receive karte time: Ribbon kholo → Box kholo → Paper hatao. Exactly reverse order! Ribbon pehle nahi khol sakte.

Why encapsulation matters: Layer 3 receiving machine reads exactly the header that Layer 3 sending machine wrote. No layer touches another layer's header!

🏛️ OSI Model — 7 Layers

- | | | |
|---|--|---------|
| 7 | Application — User + App ka interface (HTTP, DNS, FTP) Message/Data | |
| 6 | Presentation — Encryption, Compression, Format conversion | Data |
| 5 | Session — Connection manage karna, sessions | Data |
| 4 | Transport — End-to-end (TCP/UDP), reliability | Segment |
| 3 | Network — Routing, IP addresses, across networks | Packet |

2 Data Link — MAC addresses, error check, 1 LAN ke andar

Frame

1 Physical — Actual bits → electrical/optical/radio signals

Bits

*Top to Bottom: "All People Seem To Need Data Processing"**Application, Presentation, Session, Transport, Network, Data Link, Physical***TCP/IP Model — Real world yahi use hota hai!**

TCP/IP Layer	OSI Layers	Key Protocols
Application	App + Presentation + Session (5,6,7)	HTTP, HTTPS, DNS, FTP
Transport	Transport (4)	TCP, UDP
Internet/Network	Network (3)	IP, ICMP
Link/Network Access	Data Link + Physical (1,2)	Ethernet, WiFi, 4G

**OSI Layers Details****Layer 2 — Data Link (MOST IMPORTANT DISTINCTION)***MAC addresses use karta hai — within ONE LAN only**Media Access Control — "Ek baar mein ek!" — decides who transmits when on shared medium**Error checking — bits sahi pohonche ya nahi***Layer 2 vs Layer 3 — SABSE ZYADA CONFUSED HOTA HAI!***Layer 2 (Data Link) = Gali ke andar delivery — MAC address, ek LAN**Layer 3 (Network) = City se city delivery — IP address, across networks**Switch ko IP ki zarurat nahi! Router ko MAC ki zarurat nahi (usually)!***Layer 6 — Presentation:***Encryption/Decryption — TLS yahan bhi consider hota hai**Compression — zip/unzip**Character encoding — ASCII to Unicode conversions**Image encoding formats*

PDUs — Protocol Data Units

Layer	PDU Name	Yaad karne ka trick
Application / Presentation / Session	Message / Data	User data hai yaar
Transport	Segment	TCP/UDP "segment" karta hai
Network	Packet	Network "packet" bhejta hai
Data Link	Frame	Frame mein data "frame" hota hai
Physical	Bits	Actual 0s and 1s!

3

Lecture 4 Devices, Topologies & Cloud

 **Main idea:** Devices ka difference = ye packet mein kitna deep dekhte hain (which layer pe operate karte hain). Layer number = intelligence level!

Network Devices — THE Master Table (Yaad karo!)

Device	Layer	Decision Based On	Analogy
 Hub	L1 – Physical	Kuch nahi! Har port ko broadcast karta hai. Dumb device.	Megaphone 
 Switch	L2 – Data Link	MAC addresses – smart delivery ek LAN ke andar	Smart delivery boy 
 Router	L3 – Network	IP addresses – different networks ke beech routing	GPS navigator 
 Firewall	L3/L4+	IP + Port numbers. Deep inspection = aur layers. Blocks/allows traffic.	Security guard 
 Load Balancer	L4/L7	Kaunse server ko request bhejo – traffic distribute karta hai	Traffic police 

 Wireless Router/AP	L1-L3	Wireless access + routing functionality	WiFi tower 
--	-------	---	--

1234 Direct Connection Problem

Agar har computer ko har doosre computer se directly connect karo:

▸ Cables needed for n devices

$$\text{Cables} = n \times (n - 1) / 2$$

10 devices = 45 cables | 100 devices = 4,950 cables! 🤯 Isliye switches/hubs!

🏗️ Network Topologies

★ Star Topology

Sabke central hub/switch se connection.

✓ Easy setup, easy debugging

✓ Popular for LANs

✗ Central device = Single Point of Failure (SPOF)

Best for: Small office

🕸️ Mesh Topology

Har device doosre se directly connected.

✓ Very high fault tolerance

✓ Multiple redundant paths

✗ $n(n-1)/2$ cables – "spaghetti network"

Best for: Smart home, wireless mesh, critical systems

🕒 Ring Topology

Sab ek ring mein connected.

✓ Simple

✗ Ek break → puri ring fail

Purana, kam use hota hai

🔄 Hybrid Topology

Multiple topologies combine karo.

✓ Flexible, best of all

✓ Real networks yahi use karte hain

✗ Complex to manage

Best for: Data centres, large organizations

🎯 Environment → Topology: Small office → Star / Smart home → Mesh / Data centre → Hybrid

Fault Tolerance vs Scalability:

Topology	Fault Tolerance	Scalability
Star ★	Medium ⚠️	Good ✓

Mesh 	High  	Excellent  
Hybrid 	High  	Hard 

Physical → Cloud: Same concepts, different execution

Traditional (Physical)

Cloud Era (Virtual)

Physical hardware box	Virtual instance (EC2, etc.)
Copper cables	Virtualised overlays
Manual wiring	API calls / console clicks
Physical switch	VPC routing tables

 **Key insight:** Cloud mein same concepts hain — load balancer wahi hai, subnet wahi hai, AZs = fault domains. Execution different hai — software-defined. Concepts kabhi obsolete nahi hote!

AWS Deep Dive

 **Regions:** Isolated geographic areas (Mumbai, Frankfurt, etc.) with 30+ globally. Ultra-low-latency backbone se connected hain.

 **Availability Zones (AZ):** Region ke andar multiple discrete data centers — redundant power, networking. Fire/flood ek mein → doosra chal raha hai. Multi-AZ deployment = fault tolerant!

 **Edge Locations:** CDN endpoints jo user ke paas content cache karte hain → propagation delay kam hoti hai (d/s mein d kam hota hai). Netflix appliances ISP networks mein — same idea!

 **VPC:** Apna private software-defined network. IP ranges, subnets, routing control karo. Subnets → AZs se map hote hain (Layer 3 partitioning).

 Load Balancer link back to Lecture 2: Bottleneck idea! Server slow ho (bottleneck) toh zyada servers add karo aur load balancer se distribute karo. Server faster nahi kiya — zyada servers lagaye!

Lecture 5 HTTP, P2P & REST

Network Architectures

Client-Server

Powerful servers data store karte aur requests process karte hain. Multiple clients depend karte hain server pe.

Always-on servers, clients directly communicate nahi karte.

Example: YouTube, Instagram, Gmail

Peer-to-Peer (P2P)

Koi central server nahi. Har peer dono client aur server ka kaam karta hai. Highly distributed.

✓ Scales well

✗ Hard to manage

Example: BitTorrent, old Napster

HTTP — HyperText Transfer Protocol

HTTP = Application Layer ka protocol. Web ki language. Stateless hai — server ko pichle requests YAAD NAHI rehte!

 Agar stateless hai toh shopping cart kaise kaam karta hai?

Cookies & Session Tokens! Server tumhe ek token deta hai jo browser store karta hai aur har request ke saath bhejta hai. Server phir dekh ke recognize karta hai.

HTTP Request Format:

 HTTP Request Structure

[Method] [URL] [HTTP Version] ← Request Line

Host: www.example.com ← Headers

User-Agent: Mozilla/5.0

Accept: text/html

[blank line]

[body — for POST/PUT only] ← Body

HTTP Response Format:

 HTTP Response Structure

HTTP/1.1 200 OK ← Status Line

Content-Type: text/html

Content-Length: 1234

Date: Wed, 03 Sep 2026 ← Headers

[blank line]

[actual HTML/JSON/data] ← Body

🔧 HTTP Methods = CRUD Operations

Method	CRUD	Kya karta hai	Side effects?
GET	Read	Data lao, kuch change nahi hota	No ✅ (safe)
POST	Create	Naya data banao / submit karo	Yes ❌
PUT	Update	Existing data poori tarah replace karo	Yes ❌
DELETE	Delete	Data delete karo	Yes ❌
PATCH	Partial Update	Sirf kuch fields update karo	Yes ❌



HTTP/1.1 Pipelining note: Sirf safe methods (GET, HEAD, OPTIONS) pipeline ho sakte hain. POST/PUT nahi – kyunki ye state change karte hain!

📊 HTTP Status Codes

2xx ✅ Success

200

OK – Request successful

201

Created – New resource banaya

204

No Content – Success, koi body nahi

3xx 🔄 Redirect

301

Moved Permanently

302

Found (temporary redirect)

304

Not Modified (use cache)

4xx ❌ Client Error

400

Bad Request

403

Forbidden (no permission)

404

Not Found

5xx 🌟 Server Error

500

Internal Server Error

503

Service Unavailable

Client sahi tha, server mein problem!

🚀 HTTP Versions — Online Shopping Delivery Analogy

HTTP/1.0 (1996) – One item per truck 🚚

Ek file = brand new connection. Poora TCP setup baar baar! Bahut slow.

❌ No persistent connections / ❌ Ek request at a time / ❌ Very inefficient

HTTP/1.1 (1997) – Keep-Alive truck 🚚

✅ Persistent connections (Keep-Alive) – ek connection pe multiple requests

✅ Caching, Host headers

⚠️ Optional pipelining (multiple requests without waiting for response)

❌ Head-of-Line (HoL) Blocking: Responses FIFO order mein aane chahiye! Ek slow response = sab blocked!

HTTP/2 (2015) – Restaurant with multiple kitchen windows 🍽️

✅ Binary Protocol (text nahi – framing layer)

✅ Multiplexing – ek TCP connection pe multiple streams simultaneously!

✅ Server Push – server proactively resources bhej sakta hai

❌ TCP-level HoL abhi bhi hai: 1 lost TCP segment → ALL streams stall!

HTTP/3 (June 2022) – QUIC Protocol 🚀

✅ TCP ki jagah **QUIC** (Google-developed, UDP-based)

✅ Truly independent streams – ek stream loss sirf wahi stream roken

✅ HoL blocking completely solved! 🎉

✅ Faster connection setup

HTTP Versions Comparison:

Version	Connection	HoL Blocking	Protocol
HTTP/1.0	1 TCP per request	Full ❌❌	TCP, Text
HTTP/1.1	6 TCP (fake parallelism)	Full at TCP level ❌	TCP, Text
HTTP/2	1 TCP, many streams	App fixed, TCP remains ⚠️	TCP, Binary
HTTP/3	QUIC (UDP)	Fully solved ✅✅	QUIC/UDP

🔌 REST APIs

REST = Representational State Transfer. Applications ke beech communication ka architectural style.



83% public APIs REST use karte hain (Q4 2024 data)



Resources = URLs identify karte hain (eg: /users/123)



Actions = HTTP Methods (GET/POST/PUT/DELETE = CRUD)



Usually JSON format mein data exchange hota hai



Stateless – har request mein sari info honi chahiye



CRUD = Create Read Update Delete = POST GET PUT DELETE

REST = HTTP methods + meaningful URLs = Clean API design

5

Lecture 6 HTTPS & TLS — Web Security



Browser ka padlock kya guarantee karta hai?



Safe website? – NAHI! Padlock malware protect nahi karta



Encrypted connection? – Haan, data encrypt ho raha hai



Verified identity? – Haan, CA ne certificate verify kiya hai



HTTP vs HTTPS — Security difference

HTTP

Plain text – koi bhi traffic capture karke dekh sakta hai

No identity verification

MITM attack possible

Port 80

HTTPS

Encrypted – sirf sender/receiver padh sakte hain

Certificate se server identity verified

MITM certificates se blocked

Port 443

HTTPS = HTTP + TLS

Pehle SSL (Secure Socket Layer) tha – ab TLS (Transport Layer Security) use hota hai. Naam "HTTPS" rakhta hai phir bhi.

🎯 Cryptography ke 3 Core Goals



Confidentiality

Sirf authorized parties padhein.
"No eavesdropping"
Encryption se achieve hota hai



Integrity

Data modify na ho transit mein.
"No tampering"
Hashing/MACs se achieve hota hai



Authentication

Sender aur receiver ki identity verify karo.
"Who are you exactly?"
Certificates se achieve hota hai



Encryption ke Types

1. Symmetric Encryption



Ek hi key se encrypt aur decrypt karo — **same key** dono parties ke paas



Bahut **fast!** — actual data ke liye perfect



Problem: Key securely share kaise karo strangers ke saath? Agar key share karte waqt koi sun le?

2. Asymmetric Encryption (Public-Key Cryptography)



Public Key — sabko pata. Koi bhi is se encrypt kare. Public rakho.



Private Key — sirf owner ke paas. Isi se decrypt hota hai. Secret rakho!



Strangers bhi safely communicate kar sakte hain — public key se encrypt karo!



Slow — real data ke liye use nahi karte



TLS ka Key Insight — Best of Both Worlds!

Asymmetric → Handshake mein shared secret safely agree karo

Symmetric → Actual data fast encryption ke liye

Dono milake: Secure key exchange + Fast communication! 🎉



Certificates & Certificate Authorities (CA)

Problem: Kaise pata ki server genuine hai? Impostor ho sakta hai! (microsoft.com vs microsoft.com — dekho carefully!)

Digital Certificate = Server ki identity proof document. Server ka Public Key + CA ka digital signature.

Certificate Authority (CA) = Trusted third party jo certificates verify aur sign karti hai.
(DigiCert, Let's Encrypt, Comodo, etc.)

Chain of Trust:

Browser → Certificate check karta hai → CA ka signature verify karta hai → CA ko browser already trust karta hai (Root CAs pre-installed hain OS/Browser mein) → Identity confirmed!



Agar certificate invalid/expired/self-signed: → "Not Secure" warning! ⚠️

👊 TLS Handshake — Step by Step

- 1 **ClientHello:** Client kehta hai — "Main TLS 1.3/1.2 baat kar sakta hoon. Ye cipher suites support karta hoon. Ye mera random nonce (32-byte string) hai."
- 2 **ServerHello + Certificate:** Server kehta hai — "Ye cipher suite use karte hain. Ye mera certificate hai (public key + CA ka signature)."
- 3 **Certificate Validation (Authentication):** Client certificate chain validate karta hai CA se. Valid → trust! Invalid → "Not Secure" warning. ⚠️
YE STEP MITM ATTACK ROKTA HAI!
- 4 **Key Exchange:** Client aur Server milkar ek shared **session key** derive karte hain — asymmetric crypto use karke. Ab dono ke paas same symmetric key hai!
- 5 **Encrypted Communication Starts:** Ab actual data symmetric session key se encrypt hokar jaata hai. Fast + Secure! 🎉

👤 MITM (Man-in-the-Middle) Attack

Attacker tumhare aur server ke beech baithta hai — traffic capture/modify karta hai. HTTPS isko Certificate Validation se rokta hai.



MITM Defense: MITM apna khud ka certificate use karega. Browser check karega — "CA ne ye certificate sign kiya?" → No trusted CA signature → Browser: "⚠️ Connection Not Secure!" MITM fail!

🔧 HTTPS ka Layering Stack

📖 HTTPS Protocol Stack (Top to Bottom)

7. Application Layer: HTTP (actual web request/response)

↓ (HTTP jaata hai TLS ko)

TLS Layer: Encrypt karta hai, authenticate karta hai

↓

4. Transport Layer: TCP (reliable delivery)

↓

3. Network Layer: IP (routing)

↓

1-2. Link + Physical: Actual cables/WiFi

🎯 **HTTPS = HTTP + TLS = Encrypted + Authenticated + Confidential**
 Asymmetric = key exchange (handshake) / Symmetric = actual data (fast)

6

🎯 Exam Prep

MCQ Practice + Quick Cheat Sheet

🧠 Practice MCQs — KAL KE LIYE!

Q1. Packet switching mein kya hota hai jab router ka buffer full ho jaata hai?

a) Circuit establish hota hai b) Packets drop ho jaate hain c) Speed slow hoti hai d)

Encryption hota hai ☒ b) Packets drop — Packet Loss! TCP later retransmit karega.

Q2. 8000 bits ka packet, 1 Mbps link pe transmit karne mein kitna time lagega?

a) 8ms b) 0.8ms c) 80ms d) 8μs

☒ a) 8ms — $L/R = 8000 / 1,000,000 = 0.008s = 8ms$

Q3. Switch kaunsi layer pe operate karta hai aur kya use karta hai decisions ke liye?

a) L1, signals b) L2, MAC addresses c) L3, IP addresses d) L4, ports

✓ b) Layer 2 – Data Link. MAC addresses use karta hai, ek LAN ke andar.

Q4. HTTP/1.1 mein Head-of-Line blocking ka kya matlab hai?

a) Server crash b) Ek slow response sab baad wale requests ko delay kare c) Connection drop d) Packet loss

✓ b) FIFO order mandatory – ek slow/stuck response sab ko block karta hai!

Q5. TLS mein actual data ke liye symmetric encryption kyun use hoti hai, asymmetric nahi?

a) More secure b) Asymmetric slow hai c) TCP require karta hai d) HTTPS standard

✓ b) Asymmetric bahut slow hai – symmetric fast hai. Dono ka combination best hai!

Q6. OSI model mein "Segment" kis layer ka PDU hai?

a) Network (L3) b) Data Link (L2) c) Transport (L4) d) Physical (L1)

✓ c) Transport Layer (L4) – TCP/UDP ka PDU = Segment

Q7. HTTP/3 kaunsa transport protocol use karta hai aur kisne banaya?

a) TCP by IETF b) UDP directly c) QUIC over UDP by Google d) SCTP

✓ c) QUIC over UDP – Google ne banaya. HoL blocking completely solve karta hai!

Q8. Store-and-forward rule ka kya matlab hai?

a) Data store karo aur baad mein bhejo b) Router poora packet receive kare TABHI forward kare c) Buffer mein rakh ke speed control d) Forward table store karna

✓ b) Poora packet fully receive hona chahiye pehle – phir forward hota hai. Ek bit bhi pehle nahi!

Q9. HTTPS mein padlock symbol ka accurate matlab kya hai?

a) Site bilkul safe hai + no malware b) Encrypted connection + CA-verified identity c) No viruses d) Government approved

✓ b) Encrypted connection + Verified server identity. "Safe site" guarantee nahi karta!

Q10. End-to-end throughput kaise decide hoti hai?

a) Fastest link se b) Average of all links c) Slowest (bottleneck) link se d) First link se

✓ c) Bottleneck link – chain ki mazbooti = sabse kamzor kadi. $\min(R_1, R_2, \dots)$

Q11. Hub kaunsi layer pe operate karta hai aur kya karta hai?

a) L2, MAC-based forwarding b) L1, sab ports ko broadcast c) L3, IP routing d) L4, load balancing

✅ b) Layer 1 – Physical. Kuch nahi sochta, sab ports ko blindly broadcast karta hai!

Q12. Certificate Authority (CA) ka kaam kya hai HTTPS mein?

a) Packets encrypt karna b) Server certificates verify aur sign karna c) Keys generate karna d) DNS resolve karna

✅ b) CA server ki identity verify karta hai aur certificate sign karta hai. MITM rokne ke liye!

~ ~ ~ ~ ~ ~ ~ ~

⚡ FINAL CHEAT SHEET — Last 5 minutes revision

🔴 Key Formulas

Transmission:

L/R

Propagation:

d/s

Cables:

$n(n-1)/2$

Throughput:

$\min(\text{all links})$

🟡 Devices by Layer

L1 = Hub (broadcast all) 📢

L2 = Switch (MAC) 📡

L3 = Router (IP) 🌐

L3/L4 = Firewall 🛡️

L4/L7 = Load Balancer 🚦

🟣 HTTP Status Codes

2xx = Success ✅

3xx = Redirect 🔄

4xx = Client Error ❌

5xx = Server Error ⚡

404 = Not Found / 500 = Server broken

🟢 TLS Summary

Asymmetric = handshake (key share)

Symmetric = actual data (fast)

CA = certificate issuer

MITM = certificate check se blocked

🟡 HTTP Versions

🟡 OSI PDUs

1.0 = 1 request/connection ❌
1.1 = Keep-alive, HoL blocking ⚠️
2 = Binary, Multiplexing, TCP HoL ⚠️
3 = QUIC/UDP, HoL solved! ✅

App = Message/Data
Transport = Segment
Network = Packet
Data Link = Frame
Physical = Bits

🟣 Topologies

Star = central hub, SPOF
Mesh = all connected, fault tolerant
Hybrid = mixed, real networks
Small office → Star
Data centre → Hybrid

🟢 AWS Terms

Region = geographic area
AZ = fault-isolated data centres
Edge Location = CDN cache
VPC = your private network

🧠 OSI 7 to 1: All People Seem To Need Data Processing
(Application, Presentation, Session, Transport, Network, Data Link, Physical)

💪 TU KAR SAKTA HAI BHAI!

Sab kuch padh liya — ab confidently jaao!

Best of luck for tomorrow's test! 🎯 🚀

CSAI321 — Computer Networks — Master ho chuke ho!



Made with ❤️ & chai ☕ for kl ke test
CSAI321 Computer Networks — All 6 Lectures Covered 📖